



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module D: Modern Ecosystem JSON/JSONB

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

1

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

2

PURPOSE

- To understand how PostgreSQL supports **semi-structured data** with JSON and JSONB.
- To learn how to **store, query, and manipulate JSON documents** inside relational tables.
- To evaluate when JSON/JSONB is the **right tool** vs traditional relational modelling.

3

OBJECTIVES

- Differentiate** between JSON and JSONB types (text-based vs binary).
- Insert and validate** JSON data into PostgreSQL tables.
- Use JSON operators and functions** (->, ->>, #>, etc.) to extract values.
- Query nested JSON structures** with indexing (GIN, JSONB path ops).
- Compare approaches**: relational schema vs JSON/JSONB for flexible data.
- Identify use cases** (e.g., flexible metadata, APIs, logs) where JSON/JSONB improves design.

4

SEMI-STRUCTURED DATA (JSON)

- Key Value pairs
- Scalar: string, number, boolean, null
- Non-Scalar: objects, arrays
- UTF-8
- Part of ISO SQL standard with vendor specific extensions

5

WHY SEMI-STRUCTURED DATA IN RDBMS?

- Apply schema flexibility for rapidly-changing fields
- Use semi-structured data without leaving RDBMS
- Single-storage architecture: Structured and semi-structured data in one place
- ACID + SQL power meets document style (no-SQL)

6

CORE USE-CASES

- User profiles & preferences (sparse attributes)
- Event payloads, audit logs
- Third-party API responses
- Product catalogue with variable specs

Slide 7

7

DATA TYPES

- JSON data types
 - JSON and JSONB (postgres specific)
 - enforcing each stored value is valid according to the JSON rules
- JSONPATH (postgres specific)
 - Support for the SQL/JSON path language
 - SQL character string literal notation
 - Dot (.) is used for member access
 - Square brackets ([]) are used for array access
 - SQL/JSON arrays are 0-relative, unlike regular SQL arrays that start from 1

Slide 8

8

JSON VS JSONB

Feature	json	jsonb
Storage	Text	Binary decomposed
Duplicate keys	Preserved	Removed (last wins)
Key order	Preserved	Not preserved
Indexing	Very limited	GIN / GiST supported
Operator speed	Slower	Faster
Standard compliance	ISO compliant	Postgres specific

Always use JSONB in Postgres unless there is a very good reason to use plain text JSON.

Slide 9

9

JSON VS POSTGRES DATATYPES

JSON primitive type	PostgreSQL type	Notes
string	text	\u0000 is disallowed, as are Unicode escapes representing characters not available in the database encoding
number	numeric	NaN and infinity values are disallowed
boolean	boolean	Only lowercase true and false spellings are accepted
null	(none)	SQL NULL is a different concept

Slide 10

10

CRUD ACTIONS

- CREATE TABLE t(
id SERIAL PRIMARY KEY,
data **JSONB** NOT NULL);
- INSERT INTO t(data) VALUES ('{"k":"v"}'::JSONB);

Always cast to JSONB or define the column as JSONB so literals auto-cast.

Slide 11

11

CRUD ACTIONS

- Simple scalar/primitive value -- Primitive values can be numbers, quoted strings, true, false, or null
 - SELECT '5'::json;
- Array of zero or more elements (elements need not be of same type)
 - SELECT '[1, 2, "foo", null]'::json;
- Object containing pairs of keys and values -- Note that object keys must always be quoted strings
 - SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::json;
- Arrays and objects can be nested arbitrarily
 - SELECT '{"foo": [true, "bar"], "tags": {"a": 1, "b": null}}'::json;

Slide 12

12

POSTGRES SPECIFIC JSON OPERATORS

Operator	Description	Example	Supported Types	Result Type
->	Access JSON object field or array element (returns JSON).	data->'planet'	json/jsonb	json/jsonb
->>	Access JSON object field or array element as text.	data->>'name'	json/jsonb	text
#>	Access nested JSON object/array (returns JSON).	data#>'planet.details'	json/jsonb	json/jsonb
#>>	Access nested JSON object/array as text.	data#>>'planet.details.type'	json/jsonb	text
@>	Checks if left JSON contains right JSON.	data @> '{"type": "terrestrial"}'	jsonb	boolean
<@	Checks if left JSON is contained by right JSON.	data <@ '{"planet": {"moons": ["Phobos", "Deimos"]}}'	jsonb	boolean
?	Checks if key exists at top level.	data ? 'moons'	jsonb	boolean
?	Checks if any of multiple keys exist.	data ? array['name', 'type']	jsonb	boolean
?&	Checks if all specified keys exist.	data ?& array['name', 'moons']	jsonb	boolean
	Concatenates two JSONB values (merges objects or appends arrays).	data '{"discovered": true}'	jsonb	jsonb
-	Deletes key or array element by key/index.	data - 'moons'	jsonb	jsonb
#-	Deletes nested key.	data #- 'planet.details.type'	jsonb	jsonb

Slide 13

13

POSTGRES ISO COMPLIANT FUNCTIONS

Function	Description	Example Usage	Result Type
JSON_VALUE()	Extracts a scalar value from a JSON object using a JSON path.	SELECT JSON_VALUE(data, '\$.planet.name')	text or scalar
JSON_QUERY()	Extracts an object or array using a JSON path (returns structured JSON).	SELECT JSON_QUERY(data, '\$.planet.moons')	jsonb
JSON_EXISTS()	Tests if a JSON path exists in the document.	SELECT JSON_EXISTS(data, '\$.planet.type')	boolean
JSON_TABLE()	Converts JSON data into relational rows and columns.	FROM JSON_TABLE(data, '\$.planet.moons[*]' COLUMNS ...)	Table
IS JSON	Validates if a value is valid JSON.	WHERE data IS JSON	Boolean predicate
JSON_OBJECT()	Constructs a JSON object from key-value pairs.	SELECT JSON_OBJECT('name' VALUE 'Mars')	json / jsonb
JSON_ARRAY()	Constructs a JSON array from values.	SELECT JSON_ARRAY(1, 2, 3)	json / jsonb
JSON_ARRAYAGG()	Aggregates values into a single JSON array.	SELECT JSON_ARRAYAGG(name) FROM planets	json / jsonb
JSON_OBJECTAGG()	Aggregates key-value pairs into a single JSON object.	SELECT JSON_OBJECTAGG(key, value) FROM kv_table	json / jsonb

Use ISO functions for portability, and PostgreSQL JSON operators for power and performance if staying in PostgreSQL.

Slide 14

14

INDEXING

- **JSONB data type columns** → often use the **GIN index type** for efficient querying.
- A **GIN operator class** can be specified for further optimisation, depending on the query types.
 - ojson_obs: Default operator class. Supports **containment** (@>), **existence** (?)
 - ojson_path_ops: Supports only **containment** (@>) but is **smaller and faster**. Ideal for indexing when only containment queries are needed.
- A GIN index can be defined for a specific JSON document element

Slide 15

15

INDEXING SYNTAX

- Examples
 - oCREATE INDEX data_gin ON celestial_objects USING GIN (data);
 - Any containment/existence queries
 - oCREATE INDEX data_gin_path ON celestial_objects USING GIN (data jsonb_path_ops);
 - Containment only at top levels, smaller/faster index
 - oCREATE INDEX data_type_gin ON celestial_objects USING GIN (data -> 'type');
 - Optimised filter for frequent conditions on keys or when using @, ? in where clause
 - oCREATE INDEX data_terrestrial_idx ON celestial_objects ((data->>'type')) WHERE data->>'type' = 'terrestrial';
 - Optimised filter for frequent conditions (B-tree)

Slide 16

16

INDEXING STRATEGIES

- Full-document GIN: flexible but big
- Expression index on hot path: smaller & faster
- jsonb_path_ops for JSONPath search
- Generated column + B-tree for most-queried key

Always run EXPLAIN (ANALYZE) before and after indexing.

Slide 17

17

UPDATING JSONB (POSTGRES SPECIFIC)

- **jsonb_set** replaces or adds a value at a specified path in a JSONB document.
- Syntax: jsonb_set(target_jsonb, path_array, new_value_jsonb[, create_missing])
- Example (rename the planet)


```
SELECT jsonb_set('{"planet": {"name": "Mars", "moons": ["Phobos", "Deimos"]}}', {'planet.name'}, "Ares");
```
- Beware full-row rewrite for large docs

Slide 18

18

MERGING JSONB (POSTGRES SPECIFIC)

- `||` merges two JSONB values together.
 - Objects: right-hand values override left-hand keys.
 - Arrays: concatenated elements.
- Example:

```
SELECT
'{"planet":{"name":"Mars","details":{"type":"terrestrial"}}}::jsonb
|| '{"planet":{"details":{"type":"desert","gravity":"0.38g"}}}::jsonb;
SELECT '{"planet":{"moons":{"Phobos","Deimos"}}}'::jsonb ||
'{"planet":{"moons":{"Eris"}}}'::jsonb;
```

Slide 19

19

LIGHTWEIGHT VALIDATION

- a **CHECK constraint** can be used for **lightweight JSON/JSONB validation** by ensuring that certain keys or values are present and meet simple conditions.
 - Use `?` to check for key existence.
 - Use `->>` to extract values for comparison.
 - Use `jsonb_typeof` to require a specific value type for a key
- Example:

```
CREATE TABLE planets (id serial PRIMARY KEY, data jsonb NOT NULL,
CHECK ( data ? 'planet'
AND (data->>'name') IS NOT NULL
AND jsonb_typeof(data->'moons') = 'array');
```

Slide 20

20

GENERATED COLUMNS FROM JSON

- Real table column whose value is **computed** from an expression and **stored** on disk.
- When data changes, PostgreSQL recomputes the generated column values automatically.
- Syntax:

```
ADD [COLUMN] column_name data_type GENERATED ALWAYS
AS ( generation_expression ) STORED
```
- Example:

```
ADD COLUMN planet_name text
GENERATED ALWAYS AS (data #>> '{planet,name}') STORED;
```

Slide 21

21

CONVERT JSON INTO RELATION TABLE (JSON_TABLE)

- **JSON_TABLE** convert JSON arrays/objects into relational rows & columns.
- Syntax: `JSON_TABLE (<jsonb>, <json-path> COLUMNS ( <name> <type> PATH <path> [, ...] [seq FOR ORDINALITY] ) )`
- Key points
 - `FOR ORDINALITY` gives 1-based position, useful to preserve the original order of array items.
 - Can be used in `FROM`-list, views, CTEs.
 - Index with `GIN jsonb_path_ops` for heavy use

Slide 22

22

HYBRID MODELLING PATTERN

- **Pattern 1** — Core relational + sidecar JSONB
 - Stable, indexed columns for critical predicates
 - JSONB column for optional/fast-evolving fields
- **Pattern 2** — Wide sparse JSONB table
 - When attributes are truly unbounded (IoT, logs)
 - Use expression indexes for your top filters

Slide 23

23

WHEN NOT TO USE JSONB

- You query and join a field **constantly** → make it a column
- You need **strong constraints** across keys → relational design
- You do **heavy updates** to large docs → bloat, write amplification
- **Cross-row** relationships inside JSON → hard to enforce

Slide 24

24

MIGRATION STRATEGY

- Start flexible (JSONB) during rapid change
- Add generated columns & indexes for hot keys
- Gradually normalise stable attributes
- Use views to shield consumers

Slide 25

25

PERFORMANCE PITFALLS

- Unindexed predicate => seq scan
- Full GIN index on every table
- Large doc updates causing bloat
- Relying on JSONB for relational joins

Slide 26

26

KEY TAKEAWAYS

- JSONB: powerful hybrid storage
- Index smartly (expression before full GIN)
- Validate critical keys
- Measure & migrate as data stabilises

Semi-structured data is not "schemaless". It has a schema that is flexible, implicit, or self-describing rather than rigid.

Slide 27

27

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module D: Modern Ecosystem
Database Model Overview

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

28

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 29

29

PURPOSE

- To provide a broad understanding of the **different database models** developed over time.
- To show how models evolved from hierarchical and network DBs to relational to other Database models.
- To build the foundation for understanding how **data representation impacts storage, querying, and performance**.

Slide 30

30

OBJECTIVES

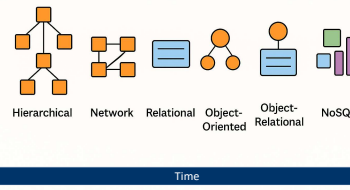
- **Describe** the main database models.
- **Compare** their key features, advantages, and limitations.
- **Explain** the historical evolution of database models and why new models emerged.
- **Identify** which models are best suited for particular use cases (e.g., transactions vs analytics).
- **Recognize** the impact of data models on query languages.

Slide 31

31

OVERVIEW DATABASE MODELS

Types of Database Models



Slide 32

32

HIERARCHICAL MODEL (1960S)

Feature	Description
Structure	Tree-like hierarchy (1-to-many parent-child)
Design Principle	Data organized in a top-down tree structure
Navigation	Navigational (parent to child)
Query Language	Procedural (e.g., IMS DML)
Example	IBM IMS DB (NASA's Apollo space program)
Strengths	Fast access for hierarchical data; simple structure
Limitations	Rigid; complex to reorganize; redundancy
Use Cases	Legacy banking systems, early payroll or HR systems

Slide 33

33

NETWORK MODEL (LATE 1960S)

Feature	Description
Structure	Graph-like, many-to-many relationships via sets
Design Principle	Records connected through owner-member sets
Navigation	Navigational, uses pointers
Query Language	CODASYL DML
Example	IDMS
Strengths	Flexible relationships; efficient for complex structures
Limitations	Complex schema; hard to maintain
Use Cases	Legacy insurance, manufacturing, telecom billing, complex enterprise systems

Slide 34

34

RELATIONAL MODEL (1970)

Feature	Description
Structure	Tables (relations) with rows and columns
Design Principle	Based on set theory and predicate logic
Navigation	Declarative
Query Language	SQL
Example	MySQL, PostgreSQL, Oracle DB
Strengths	Simplicity, data integrity, ad-hoc queries
Limitations	Poor performance on complex objects or nested data
Use Cases	Enterprise apps, finance, CRM, ERP, OLTP

Slide 35

35

OBJECT-ORIENTED MODEL (1980S)

Feature	Description
Structure	Objects with attributes and methods
Design Principle	Data and behavior encapsulated in objects
Navigation	Through object relationships (like programming)
Query Language	OQL (Object Query Language)
Example	db4o, ObjectDB
Strengths	Ideal for complex applications like CAD, simulations
Limitations	Less standardization; limited support in industry
Use Cases	Scientific computing, multimedia, engineering apps

Slide 36

36

OBJECT-RELATIONAL MODEL (1990S)

Feature	Description
Structure	Relational tables with support for complex data types
Design Principle	Extend relational model with object-oriented features
Navigation	Declarative (SQL with object extensions)
Query Language	Extended SQL with object support
Example	PostgreSQL, Oracle (with object features)
Strengths	Balance of structure and flexibility; SQL compatibility
Limitations	Higher complexity; underutilized features
Use Cases	GIS, financial apps, systems with custom data types

Slide 37

37

DATA WAREHOUSE (1990S)

Feature	Description
Structure	Denormalised, column-oriented
Design Principle	Analytical, read-optimized
Navigation	OLAP, complex queries
Query Language	SQL, BI tools
Example	Snowflake, Amazon Redshift, BigQuery
Strengths	Fast analytics on large datasets
Limitations	Expensive, batch-oriented
Use Cases	BI, reporting, historical analytics

Slide 38

38

IN-MEMORY (LATE 1990S)

Feature	Description
Structure	RAM-based storage
Design Principle	Optimised for speed
Navigation	SQL or key-based
Query Language	SQL or API calls
Example	Redis, SAP HANA, MemSQL
Strengths	Extremely fast read/write
Limitations	Cost of memory, persistence options
Use Cases	Caching, real-time pricing, gaming, trading apps, SAP ERP

Slide 39

39

NOSQL (2000S)

Feature	Description
Structure	Flexible schema: key-value, documents, columns, or graphs
Design Principle	Scalability, availability, and partition tolerance
Navigation	Depends on model: key lookups, paths, graph traversal
Query Language	Varies (MongoDB Query, CQL, Gremlin, etc.)
Example	MongoDB, Cassandra, Redis, Neo4j
Strengths	High performance, scalability, schema flexibility
Limitations	Eventual consistency, learning curve, fragmented ecosystem
Use Cases	Real-time analytics, social networks, IoT, personalization

Slide 40

40

OTHER MODELS

Model	Key Features	Example
Multimodel	Supports multiple models in one engine	ArangoDB, OrientDB, Azure Cosmos DB
NewSQL	RDBMS + scalability of NoSQL	VoltDB, Google Spanner

Slide 41

41

KEY TAKEAWAYS

- DBMS evolution aligns with growing data complexity and scale.
- Relational databases remain foundational.
- Object and NoSQL models address limitations in flexibility and scale.
- Hybrid models like PostgreSQL now bridge SQL and JSON/NoSQL worlds.
- Selection depends on data structure, consistency needs, scalability, and access patterns.

Slide 42

42



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module D: Modern Ecosystem Object Relational

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

43

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

44

PURPOSE

- Introduce PostgreSQL's **object-relational features** that extend beyond standard relational databases.
- Understand how **inheritance, user-defined types, and polymorphism** can be applied in database design.
- Show how PostgreSQL bridges the gap between **relational data models** and **object-oriented programming principles**.
- Provide tools for modeling **complex, hierarchical, and extensible data structures**.

45

OBJECTIVES

- Explain** PostgreSQL's object-relational nature compared to pure relational databases.
- Use** table **inheritance** to model hierarchies.
- Define** custom **types, domains, and composite types** for reusable structures.
- Leverage** functions, operators, and overloading for polymorphic behaviour.
- Evaluate** when object-oriented features improve modelling vs when relational design is simpler.

46

ORDBMS : FEATURES

- Extension of basic data types
 - Complex objects
- Inheritance
 - Inheritance of data
 - Inheritance of types
- Rules (e.g.)
 - Update rule
 - Query update rule
 - Update query line
 - Query line
 - ...

47

INTRODUCTION

- More complex and specialised data types
 - => self-defined data types

48

SELF-DEFINING DATA TYPES

- CREATE TYPE : creates data type
- DROP TYPE : removes data type
- ALTER TYPE : feasible adjustments
- Eg
CREATE TYPE coordinate
AS (x int, y int);
- -- composite

Slide 49

49

ACCESS TO DATA TYPES

- Owner of data type : the one who creates type
- Anyone who wants to use the type must be granted permission:
GRANT usage
ON TYPE money
TO Jim
- Removal of authorisation :
REVOKE usage
ON TYPE money
FROM Jim
- See also <https://www.postgresql.org/docs/current/sql-createtype.html>

Slide 50

50

CASTING OF VALUES

- Casting : to compare different data types
- Destructor : transforms self-defined datatype to base datatype
- Constructor : transforms base datatype to self-defined datatype

Slide 51

51

EXAMPLES

```
select *  
from fine  
where amount > money(50) ;
```

Amount is of
type of money

```
select *  
from fine  
where decimal(amount) > 50 ;  
  
insert into city (where,name) values  
(coordinate(12,57), 'nowhere') ;
```

Slide 52

52

DEFINING CUSTOM OPERATORS

- Preliminary remarks :
 - Operators have been added for convenience
 - Operators belong to each basic data type
 - Operators have a meaning depending on the data type
 - Defining operators on self-defined data types ~ operations on underlying type

Slide 53

53

EXAMPLE

- Defining operators and associated functions :

```
CREATE OR REPLACE FUNCTION plus_vector(in_c1 coordinate, in_c2 coordinate)  
RETURNS coordinate AS  
$$  
DECLARE  
    result coordinate;  
BEGIN  
    result.x := in_c1.x + in_c2.x;  
    result.y := in_c1.y + in_c2.y;  
    RETURN result;  
END;  
$$  
LANGUAGE plpgsql;
```

54

Slide 54

54

EXAMPLE

- Associated operator :

```
create operator + (  
  leftarg = coordinate,  
  rightarg = coordinate,  
  function = plus_vector,  
  commutator = +  
);
```

Slide 55

55

EXAMPLE

- Usage :

```
select (1,10)::coordinate + (20,200)::coordinate as vector_sum;  
select cast((1,10) as coordinate) + cast((20,200) as coordinate)  
as vector_sum;
```

```
vector_sum  
-----  
(21,210)  
(1 row)
```

Slide 56

56

OPAQUE DATA TYPE

- Def. : data type that does not depend on a base datatype
- Defining functions to work with this is necessary
- Example:

```
create type tweedim (internallength = 4) ;  
-- base types in postgresql
```

Slide 57

57

CREATE TYPE

- ISO :
 - Composite (e.g. coordinate)
 - Opaque (pg : base)
- Not present in ISO, postgres specific :
 - Enum (enumeration, M/F/X)
 - Range
 - + Array

Slide 58

58

NAMED ROW DATA TYPE

- Def. : grouping of values that logically belong together (idem composite)
- Example:

```
create type address as  
  (street char(15) not null,  
   house_number char(4) ,  
   postal_code char(6) ,  
   city char(10) not null) ;
```
- postgresql : without not null constraint

Slide 59

59

EXAMPLE

```
create table player  
  (player_nr integer primary key,  
   ...  
   Home_address address ,  
   postal_address address ,  
   holiday_address address ,  
   ... ) ;  
  
select player_nr, home_address  
from player  
where (home_address).place = 'Leuven' ;
```

Slide 60

60

UNNAMED ROW DATA TYPE

- Unnamed row data type : : grouping of values that logically belong together without giving a name
- Example:

```
create table player
(player_nr smallint primary key,
...
Home_address row (street char(15) not null,
house_number char(4),
postal_code char(6),
city char(10) not null),
phone char(10),
... );
```
- -- postgresql : unnamed rows are only allowed as input, not in DDL

Slide 61

61

TYPED TABLE

- Def. : assign a data type to a table
- Easy to create similar tables
- Eg. create type t_player as

```
(player_nr integer not null,
name char(15) not null,
...
association_nr char(4));
```

create table player of t_player
(primary key player_nr);

Slide 62

62

TABLE > TABLE

- Adopt basic structure an existing table (fk's do not move with it!)
- Example:

```
create table new_customers
(like space_travel.customers including indexes);
```
- -- see including option for different options

Slide 63

63

INTEGRITY RULES ON DATA TYPES

- Restrictions on allowed values
- Example:

```
create type number_sets as smallint
check (value in (0, 1, 2, 3));
```
- create table match

```
(match_nr integer primary key,
team_nr integer not null,
player_nr integer not null,
nr_sets_won number_sets not null,
nr_sets_lost number_sets not null);
```
- -- What is a DOMAIN in RDBMS?
- -- PG : via enum

Slide 64

64

KEYS AND INDEXES

- Is fully analogous with self-defined data types
 - Operator needed for index must be able to be used
- For named row data types (composite):
 - at full value
 - on part of it
 - CREATE INDEX coordinate_x_idx ON your_table((coord_column).x);

Slide 65

65

INHERITANCE, REFERENCES, COLLECTIONS

- Inheritance of data types
- Linking tables via row identifiers
- Collections
- Inheritance of tables
- Rules

Slide 66

66

INHERITANCE OF DATA TYPES

- Def. : all properties of one data type are inherited by another (supertype and subtype)
- Example:

```
create type address as
(
  street      char(15) not null,
  house_number char(4),
  postal_code char(6),
  city        char(10) not null);

create type foreign_address as
(
  country char(20) not null) under address ;
```
- -- postgresql : ..or directly or like in base type....

Slide 67

67

LINKING TABLES

- In OO-DB : all rows have a unique identifier (by the system)
- REF : to retrieve identification
- Example.

```
select ref(player_nr)
from player
where player_nr = 6 ;
```
- -- postgresql : possibly via oids, little used

Slide 68

68

EXAMPLE

- REF : to link tables (not in pg)
- Example:

```
create table team
(
  team_nr smallint primary key,
  player ref(player) not null,
  division char(6) not null);

insert into team (team_nr player, division)
values (3, (select ref(player)
from player
where player_nr = 112), 'first') ;

select team_nr, player.last_name
from team;
```

Slide 69

69

PRO-CONTRA

- Advantages :
 - Always the right data type with the referenced key
 - If primary keys are wide, working with reference columns saves storage space
 - When changing primary keys, no time is lost by changing the referring key
 - Certain selects become easier
- Disadvantages :
 - Certain mutations are harder to define
 - References work in one direction
 - With DB design, one gets multiple choices, so this becomes more difficult
 - References cannot guard the integrity of data in the way that referencing keys can

Slide 70

70

COLLECTIONS

- Collections : collection of values in one cell
- Example

```
create table player
(
  player_nr smallint primary key,
  ...
  phones setof(char(13)),
  association_nr char(4));

insert into player (player_nr, ..., phones, ...)
values (213, ..., {'016-342654', '0475-654387'}, ..);

select player_nr
from player
where '016-342654' in (phones);
```
- -- pg: mostly via arrays[] ...

Slide 71

71

INHERITANCE OF TABLES

- Def. : all properties of one table are inherited by another table (supertable - subtable)
- Limitations :
 - No cyclical structure

Slide 72

72

EXAMPLE

```
create table player as
(player_nr      smallint not null,
 name          char(15) not null,
 ...
 association_nr char(4));
create table old_players
(left_date not null)
inherits (player, okra);

SELECT *
FROM (ONLY) player;
-- take care with inserts, constraints ...
-- postgresql
```

Slide 73

73

RULES

- Go beyond triggers : SELECT, INSERT, UPDATE, DELETE
- FKs >> triggered
- Updateable views
 - rules of triggers
- But carefully, system logic

Slide 74

74

CREATE RULE (E.G.)

```
CREATE RULE "Myrule" AS
ON SELECT TO match
WHERE player_nr = 7
DO INSTEAD
SELECT 'first walk around the table for three
tours and then try again';
```

Slide 75

75

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module D: Modern Ecosystem
SQL/MED

TI / BCS

Attribution-NonCommercial-ShareAlike 4.0 Unported Licence

2025-2026

76

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 77

77

PURPOSE

- To introduce **SQL/MED (Management of External Data)** and PostgreSQL's **Foreign Data Wrappers (FDW)**.
- To show how PostgreSQL can query and join data from **external sources** (other databases, files, APIs) as if they were local tables.
- To enable integration of heterogeneous data into a single queryable interface.

Slide 78

78

OBJECTIVES

- **Define** SQL/MED and the concept of Foreign Data Wrappers in PostgreSQL.
- **Explain** the difference between local tables and foreign tables.
- **Create** a foreign server and foreign table using CREATE EXTENSION, CREATE SERVER, and CREATE FOREIGN TABLE.
- **Access remote data** (e.g., from CSV files) via FDW.
- **Perform joins** between local and foreign tables seamlessly.
- **Evaluate** use cases and limitations of FDWs (e.g., performance, pushdown of queries).

Slide 79

79

SQL/MED

- **SQL/MED** stands for **SQL Management of External Data**.
- It is part of the SQL standard.
- Defines how a database system can **access data stored outside of itself**.
- Goal: make external data look like it's part of the SQL database, with a consistent query language (SQL).

Slide 80

80

SQL/MED CONCEPTS

- **Foreign-data wrapper (FDW)**
 - A driver/adaptor that knows how to communicate with an external system.
 - PostgreSQL implements this directly (e.g., postgres_fdw, mysql_fdw, file_fdw).
- **Foreign server**
 - Represents the remote system (e.g., a CSV file or RDBMS instance).
- **User mapping**
 - Stores authentication info (username/password) for connecting to the foreign server.
- **Foreign table**
 - A table definition inside your DB that points to the external data source.
- **Datalink**
 - Data type (column type) defined in SQL/MED.
 - Stores a reference (link) to an external file/object (e.g., a PDF in a filesystem).
 - Not supported in PostgreSQL –see <https://github.com/lacanoid/datalink>

Slide 81

81

USE CASES

- **Data integration across systems**
 - Query and join data from multiple databases without ETL.
 - Combine internal operational data with external reference or master data.
- **Federated queries / data virtualization**
 - Run queries across several data sources as if they were a single database.
 - Useful in reporting, analytics, and dashboards.
- **Incremental migration**
 - Move from one database to another gradually.
 - Query the old system via FDW while migrating data into Postgres.
- **Access to non-database sources**
 - Use FDWs for CSV files, JSON, or APIs.
- **Operational convenience**
 - Users can access external data without leaving Postgres.
 - Single SQL interface for multiple systems.

Slide 82

82

LIMITATIONS

- **Performance overhead**
 - Every remote call introduces network latency.
 - Large joins or aggregations may be very slow compared to local queries.
- **Pushdown limitations**
 - Not all operations (filters, joins, aggregates) are pushed down to the remote system; sometimes the data must be pulled into Postgres first.
- **Read-only in many cases**
 - Writes are supported in some FDWs (like postgres_fdw) but not universally.
- **No true distributed transactions**
 - no global ACID.

Slide 83

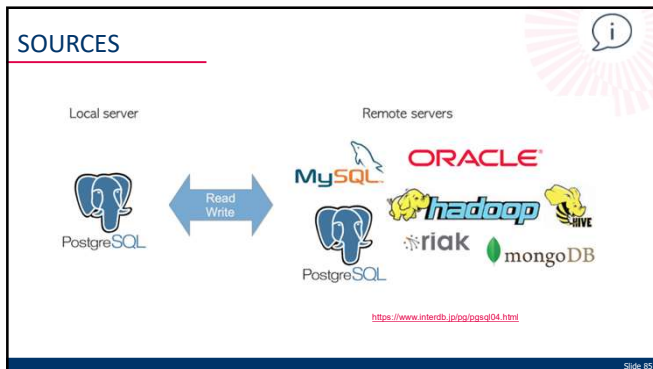
83

STORAGE VS NETWORK



Slide 84

84



85

SOURCES

- https://wiki.postgresql.org/wiki/Foreign_data_wrappers
 - Databases
 - Files
 - GIS
 - LDAP
 - Web
 - BS
 - ..

Slide 86

86

BASIC IMPLEMENTATION

- **Required for use** (library): CREATE EXTENSION ...
- **Where/how** (endpoint): CREATE SERVER ...
- **Who** (credentials): CREATE USER MAPPING ...
- **What** (tables): CREATE FOREIGN TABLE ... / IMPORT FOREIGN SCHEMA ...
- **Allow use**: GRANT USAGE/SELECT ...

Slide 87

87

EXAMPLE

- Question: Integrate the observations of celestial objects from a CSV file (objectname, obs_date, obs_time, obs_notes) into the celestial objects database. The file is stored on the DB server.
- Create the extension
`CREATE EXTENSION IF NOT EXISTS file_fdw;`
- Create the server
`CREATE SERVER csv_server FOREIGN DATA WRAPPER file_fdw;`

Slide 88

88

EXAMPLE

- Create the foreign table

```
CREATE FOREIGN TABLE space_travel.observations_csv (
  objectname text,
  obs_date date,
  obs_time time,
  obs_notes text
)
SERVER csv_server OPTIONS (
  filename './observations.csv', format 'csv', header 'true', delimiter ',',
  null '', quote '', escape ''
); -- Filename must be an absolute path on the DB server
```
- Grant permissions

```
GRANT USAGE ON FOREIGN SERVER csv_server TO user;
GRANT SELECT ON space_travel.observations_csv TO user;
```

Slide 89

89

EXAMPLE EXTENSIONS

- <https://gdal.org/>
- <https://github.com/pramsey/pgsql-ogr-fdw>

Slide 90

90



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module D: Modern Ecosystem

Cloud Managed DB Basics

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

91

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 92

92

PURPOSE

- To understand what **Cloud Managed Databases** are and why organizations adopt them.
- To explore how cloud providers (AWS RDS, Azure Database, GCP Cloud SQL, etc.) simplify database operations.
- To evaluate trade-offs between **control vs convenience** in managed services.

Slide 93

93

OBJECTIVES

- Explain the difference between **self-managed** and **cloud-managed** databases
- Identify major cloud providers
- Understand **responsibilities**: what the cloud manages vs what you still manage (shared responsibility model)
- Recognize the benefits (**scalability, HA, automation**) and challenges (**vendor lock-in, limited tuning**)
- Evaluate when to choose a **managed service** vs a **self-hosted database**

Slide 94

94

WHAT IS A CLOUD-MANAGED DATABASE?

- A database service provided and maintained by a cloud provider.
- Eliminates the need for manual infrastructure setup.
- Offers scalability, high availability, and maintenance automation.
- Examples: Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL.*

Slide 95

95

KEY CHARACTERISTICS

- Fully Managed**: Automated backups, updates, patching, compliance (GDPR, SOX, GxP,...).
- Scalable**: Easy to scale storage and compute resources.
- High Availability**: Built-in failover and replication.
- Secure**: Encryption, IAM roles, audit logging.

Slide 96

96

BENEFITS

- **Reduced Operational Overhead:** Focus on data/application logic.
- **Faster Deployment:** No hardware provisioning delays.
- **Cost Efficiency:** Pay-as-you-go pricing.
- **Global Reach:** Deploy in multiple regions for low latency.

Slide 97

97

TYPES OF CLOUD DATABASES

- **Relational:** PostgreSQL, MySQL, SQL Server
- **NoSQL:** MongoDB Atlas, DynamoDB, Cosmos DB
- **In-Memory:** Redis, Memcached
- **Data Warehouse:** BigQuery, Snowflake, Redshift

Slide 98

98

DEPLOYMENT MODELS

- **Public Cloud:** Hosted by third-party provider.
- **Private Cloud:** Dedicated infrastructure, more control.
- **Hybrid Cloud:** Combines on-prem and cloud databases.

Slide 99

99

COMMON USE CASES

- Web and mobile apps
- Real-time analytics
- Multi-tenant SaaS platforms
- IoT and telemetry data processing

Slide 100

100

LIMITATIONS

- Vendor Lock-in
- Limited Customisation
- Latency (compared to on-prem)
- Cost for High IOPS (Input/Output Operations Per Second) or Storage

Slide 101

101

MAJOR CLOUD DATABASE PROVIDERS

- **Global hyperscalers**
 - Amazon Web Services (AWS)
 - Google Cloud Platform (GCP)
 - Microsoft Azure

A **hyperscaler** is a company that provides **massive-scale cloud infrastructure and services**, capable of operating **globally distributed data centres** with **automated, elastic scalability**.

Slide 102

102

EUROPEAN CLOUD DATABASE PROVIDERS (2025)

Provider	Country	Managed DBs	Strengths	Use Cases
OVHcloud	France	PostgreSQL, MySQL, MongoDB, Redis, Cassandra	Sovereign cloud, global reach, public sector	Enterprise, public sector, SaaS
Aiven	Finland	PostgreSQL, Kafka, MySQL, Cassandra, Redis	Multi-cloud DBaaS, DevOps focus	SaaS, scalable data infrastructure
Scaleway	France	PostgreSQL, MySQL, Redis	Transparent pricing, European sovereignty	Startups, developers, SMEs
Exoscale	Switzerland	PostgreSQL (via integration)	Swiss neutrality, privacy-focused	Regulated sectors: finance, healthcare
Clever Cloud	France	PostgreSQL, MySQL, MongoDB, Redis, Elastic	CI/CD friendly, developer-first	Web apps, agile development
Stackit	Germany	PostgreSQL, MySQL, Redis	Backed by Lidl/Kaufland, EU sovereign cloud	Enterprise cloud modernization
T-Systems	Germany	PostgreSQL, SAP HANA, MS SQL (managed)	Enterprise-grade, trusted by government	Government, healthcare, large enterprise

Slide 103

103

HYPERSCALER VS CLOUD PROVIDER

Feature	Cloud Provider	Hyperscaler
Scale	Small to large	Very large (millions of workloads)
Infrastructure	Regional or national	Global (many data centers worldwide)
Automation	Varies (manual to semi-automated)	High (full autoscaling, self-service)
Elasticity	May be limited	Built for elastic scaling at massive levels
Compliance focus	Local/sovereign (GDPR, EU-only)	Global, may need region-specific adaptation
Use Case Suitability	Ideal for EU-specific or regulated needs	Ideal for global, distributed workloads

A **hyperscaler** is a specific type of **cloud provider**.

Slide 104

104

KEY TAKEAWAYS

- Cloud-managed DBs simplify database operations.
- Ideal for modern, scalable, and distributed applications.
- Choose based on workload, compliance, and cost needs.

Slide 105

105